Name: Sanjeev Kumar Patel **Enrollment No.:** 2024BCSE022

Lab 7

P1: Demonstrate operator overloading by adding two points $p_1(x_1, y_1) + p_2(x_2, y_2)$

```python
# P1: Demonstrate operator overloading by adding
# two points p1(x1,y1) + p2(x2,y2)
class Point:
    def __init__ (self, x, y):
        self.x = x
        self.y = y
    def __add__(self, nextPoint):
        return Point(self.x + nextPoint.x, self.y +
            nextPoint.y)
    def display(self):
        print("Point: ", f"({self.x}, {self.y})")
p1 = Point(2,5)
p2 = Point(3,6)
p3 = p1 + p2
p3 = p1 + p2
print("First Point:")
p1.display()
print("Second Point:")
p2.display()
print("After Addition:")
p3.display()
```

Output:

```
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7> python '.\
    P1.py'
First Point:
Point:  (2, 5)
Second Point:
Point:  (3, 6)
After Addition:
Point:  (5, 11)
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7>
```

Name: Sanjeev Kumar Patel **Enrollment No.:** 2024BCSE022

Lab 7

P2: Add two fractions $f_1 = (x_1/y_1)$, $f_2 = (x_2/y_2)$

```python
# P2: Add two fractions f1 = (x1/y1), f2 = (x2/y2)

from math import gcd
class Fraction:
    def __init__(self, numerator, denominator):
        self.numerator = numerator
        self.denominator = denominator
    def __add__(self, other):
        num = (self.numerator * other.denominator) + \
              (other.numerator * self.denominator)
        den = self.denominator * other.denominator
        g = gcd(num, den)
        return Fraction(num // g, den // g)
    def display(self):
        print(self.numerator, "/", self.denominator)

f1 = Fraction(1, 2)
f2 = Fraction(1, 3)

f3 = f1 + f2

print("First Fraction:")
f1.display()

print("Second Fraction:")
f2.display()

print("Addition:")
f3.display()
```

Output:

```
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7> python '.\
    P2.py'
First Fraction:
1 / 2
Second Fraction:
1 / 3
Addition:
5 / 6
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7>
```

Name: Sanjeev Kumar Patel **Enrollment No.:** 2024BCSE022

Lab 7

P3: Demonstrate basic regular expressions by using search(), match(), split(), findall(), group()

```python
# P3: Demonstrate basic regular expressions by using search(), match(), split(), findall(),
# group()

import re

text = "My phone number is 9876543210 and my age is 20."

# search()
result = re.search(r"\d+", text)
print("search():", result.group())

# match()
result = re.match(r"My", text)
print("match():", result.group())

# split()
result = re.split(r"\s+", text)
print("split():", result)

# findall()
result = re.findall(r"\d+", text)
print("findall():", result)

# group()
result = re.search(r"phone number is (\d+)", text)
print("group():", result.group(1))
```

Output:

```
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7> python '.\P3.py'
search(): 9876543210
match(): My
split(): ['My', 'phone', 'number', 'is', '9876543210', 'and', 'my', 'age', 'is', '20.']
findall(): ['9876543210', '20']
group(): 9876543210
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7>
```

Name: Sanjeev Kumar Patel **Enrollment No.:** 2024BCSE022

Lab 7

P4: Demonstrate advanced regular expressions for validation

```python
# P4: Demonstrate advanced regular expressions for validation

import re

# a) Validate Mobile Number

phone = input("Enter mobile number: ")

if re.fullmatch(r"^[6-9][0-9]{9}$", phone):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")


# b) Validate Landline Number

landline = input("Enter landline number: ")

if re.fullmatch(r"^[0-9]{2,4}-[0-9]{6,8}$", landline):
    print("Valid Landline Number")
else:
    print("Invalid Landline Number")


# c) Validate Email ID

email = input("Enter email ID: ")

if re.fullmatch(r"^[A-Za-z0-9.\_%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$", email):
    print("Valid Email ID")
else:
    print("Invalid Email ID")


# d) Validate Password

password = input("Enter password: ")

if (len(password) >= 8 and
    re.search(r"[A-Z]", password) and
    re.search(r"[a-z]", password) and
```

```
    re.search(r"[0-9]", password) and
    re.search(r"[^A-Za-z0-9]", password)):
    print("Strong Password")
else:
    print("Weak Password")
```

Output:

```
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7> python '.\
    p4(a).py'
Enter mobile number: 898989832
Invalid Mobile Number
Enter landline number: 3219883304322
Invalid Landline Number
Enter email ID: sanjeev2005p.05@gmail.com
Valid Email ID
Enter password: heloopyH9090
Weak Password
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7>
```

Name: Sanjeev Kumar Patel **Enrollment No.:** 2024BCSE022

Lab 7

P5: Demonstrate the Diamond Problem in Multiple Inheritance

```python
class Person:

    def show(self):

        print("Class Person")

class Mother(Person):

    def show(self):

        print("Class Mother")

class Father(Person):

    def show(self):

        print("Class Father")

class Child(Mother, Father):

    pass

obj = Child()

obj.show()
```

Output:

```
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7> python .\
    diamondProblem.py
Class Mother
PS S:\SANJEEV PATEL\College\3rd year\Python\Lab7>
```